Laboratorio de arquitectura de software

Biblioteca web en Java 21 + Maven

Guía de trabajo para estudiantes

Duración estimada: 100–120 minutos

1. Propósito de la actividad

Esta práctica conecta los conceptos de **capas lógicas, MVC Web, monolito modular y comunicación por eventos** con una aplicación que se puede ejecutar y probar. Se proporciona un **proyecto inicial compilable** con tareas pendientes, así como pruebas JUnit 4 que especifican el comportamiento esperado. Las pruebas que fallan inicialmente *no son un error de distribución*: indican código aún por implementar.
Objetivos verificables: (i) ubicar cada responsabilidad en su capa; (ii) implementar reglas de negocio fuera del controlador; (iii) responder con códigos HTTP coherentes; (iv) probar con JUnit 4 y Mockito; (v) explicar la diferencia entre eventos locales y microservicios.

2. Contexto: biblioteca

La aplicación permite consultar un catálogo de libros y registrar reservas. Cada libro posee un identificador, título y stock no negativo. Una reserva tiene un identificador, el libro asociado y la cantidad reservada. Para registrar una reserva, deben cumplirse las reglas:

R1. `cantidad` debe ser positiva.

R2. El libro debe existir; en caso contrario, se rechaza la operación.

R3. No se permiten reservas que excedan el stock disponible.

R4. Una operación exitosa descuenta stock, persiste una reserva y publica un evento `ReservaCreada`.

R5. El fallo de un observador de notificaciones no debe invalidar una reserva ya confirmada en este ejercicio.

Invariante principal. Si s_i es el stock del libro i, debe mantenerse $s_i \geq 0$ tras cualquier operación. En particular, para cantidad $q > 0$, la transición válida es

$$(s_i, q) \longmapsto \begin{cases} s_i - q, & s_i \geq q, \\ s_i, & s_i < q \quad \text{(operación rechazada)}. \end{cases}$$

La comprobación y el descuento deben ser **atómicos**: si dos solicitudes comprueban el stock por separado, ambas podrían creer que existe disponibilidad.

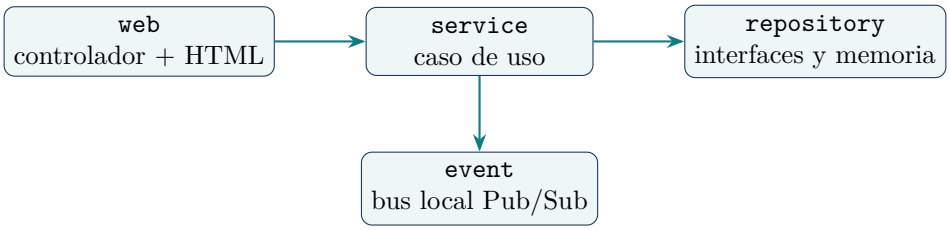
3. Preparación del proyecto

Requisitos: JDK 21, Apache Maven 3.9 o superior e IntelliJ IDEA o VS Code. Abrir el directorio `starter_estudiantes` como proyecto Maven y ejecutar:

```
java -version
mvn -version
mvn test
mvn -q -DskipTests package
mvn exec:java
```

Abrir http://localhost:8080/. La página de catálogo funciona; el envío inicial de reservas responde **501** hasta terminar las actividades. Para cambiar de puerto: `mvn exec:java -Dexec.args="8090"`. Las dependencias de pruebas se descargan la primera vez si no están en la caché Maven.

4. Organización arquitectural



- **Modelo:** `model.Libro`, `model.Reserva`.
- **Vista:** `web.VistaHtml`, generador de HTML del catálogo y reservas.
- **Controlador:** `web.BibliotecaWeb`, adaptador de solicitudes/respuestas HTTP.
- **Servicios:** `service.ReservaService`, validación y coordinación de reserva.
- **Repositorios:** interfaces + implementaciones `*EnMemoria`.

Pregunta 1. Explique por qué esta es una aplicación *monolítica modular* y por qué tener varios paquetes no significa tener microservicios.

5. Actividades de programación

Actividad A. Reglas del negocio (35 min)

Abra `service/ReservaService.java`. Implemente el método `crear(int libroId,int cantidad)` en el lugar marcado `TODO`. Su implementación debe:

a) rechazar cantidades no positivas usando `IllegalArgumentException`;

b) consultar `catalogo.buscar(libroId)` y lanzar `LibroNoEncontrado` si procede;

c) invocar el método de descuento atómico del repositorio; si retorna `false`, lanzar `StockInsuficiente`;

d) registrar usando `reservas.guardar(libroId,cantidad)`;

e) publicar `new ReservaCreada(...)` mediante `eventos.publicar`; finalmente devolver la reserva creada.

Restricción: no ubique esta lógica en la capa `web`. El repositorio ya proporciona el descuento atómico necesario para evitar stock negativo, incluso con solicitudes concurrentes.

Actividad B. Adaptador HTTP (25 min)

Abra `web/BibliotecaWeb.java`. Complete `crearReserva(HttpExchange ex, ReservaService servicio)`. Use `leerFormulario()` y la vista `VistaHtml`. La respuesta debe distinguir:

Situación	Código HTTP
Reserva creada	`201 Created`
Parámetros ausentes, no enteros o cantidad inválida	`400 Bad Request`
Libro inexistente	`404 Not Found`
Stock insuficiente	`409 Conflict`

No implemente descuento de stock en el controlador. Para probar manualmente:

```
curl -i http://localhost:8080/
curl -i -X POST http://localhost:8080/reservas \
 -H 'Content-Type: application/x-www-form-urlencoded' \
 -d 'libroId=1&cantidad=2'
curl -i http://localhost:8080/reservas
```

Actividad C. Publicación/suscripción local (20 min)

Identifique `BusEventosLocal`, `PublicadorEventos`, `SuscriptorReserva` y `ReservaCreada`. La aplicación ya registra dos suscriptores en `Main`: simulación de correo y auditoría.

a) Confirme que una reserva exitosa dispara ambos mensajes de consola.

b) Agregue un tercer suscriptor: *métricas* (incrementar un contador de reservas).

c) Explique por qué este evento **no es asíncrono ni persistente**: todos los suscriptores ejecutan en el mismo proceso.

d) Ejecute la prueba que simula el fallo de un suscriptor y explique su resultado.

Actividad D. Pruebas y justificación (20–25 min)

Ejecute `mvn test` e identifique la prueba que falla antes de corregirla. El conjunto proporcionado incluye pruebas de negocio, integración HTTP, Mockito, fallo del observador y concurrencia. Adjunte una salida final de pruebas satisfactorias. Dibuje dependencias entre los paquetes y responda: *¿qué dos cambios serían más costosos si se mezclasen reglas de stock y HTTP?*

6. Desafío de diseño: de monolito a microservicios (opcional)

Suponga que el catálogo cambia muy frecuentemente y recibe diez veces más solicitudes que el módulo de reservas. Diseñe dos procesos desplegables de manera independiente: **Servicio Catálogo** y **Servicio Reservas**. Indique (i) contrato API mínimo, (ii) propietario de stock, (iii) propietario de reservas, (iv) tratamiento de timeout, (v) mecanismo para no duplicar reservas ante reintentos. No es necesario implementar un broker real.

7. Evidencias y rúbrica

Criterio	Peso
Reglas de negocio y preservación del stock	35 %
Pruebas unitarias y de integración ejecutadas	25 %
Separación MVC / capas y contratos de repositorio	25 %
Justificación arquitectural y análisis de microservicios	15 %

Entregar: proyecto Maven, captura/salida de pruebas y un PDF breve (máximo dos páginas) con diagrama, respuesta a la Pregunta 1 y análisis del desafío de diseño.

8. Preguntas de salida

1. ¿Por qué no debe estar la regla de stock en el controlador HTTP?
2. ¿En qué se diferencia un evento local de uno entregado por un broker con reintentos?
3. ¿Por qué usar Maven, varios paquetes y Java **no** convierte una aplicación en microservicios?

Nota docente: El proyecto incluye una referencia resuelta separada para corrección. El almacenamiento es solo en memoria y no sustituye transacciones de base de datos; el servidor `jdk.httpserver` se utiliza para evitar dependencias innecesarias en el taller.